

GUIA PRÁTICO · CONSTRUIR COM IA

10

skills para sair do zero ao app

O caminho completo para quem nunca programou construir, com Claude ou Codex, o aplicativo que a sua empresa usa todo dia.

Do zero ao app

10 skills instaláveis + o método

Edição 1

O que você tem em mãos

DUAS COISAS, NÃO UMA

Este material é um **guia** e um **conjunto de ferramentas**. O guia são os capítulos que você está lendo: o método, as decisões, as armadilhas. As ferramentas são **10 skills instaláveis** — arquivos que você copia para dentro do Claude Code e que fazem o assistente conduzir cada etapa do jeito certo, sem você precisar lembrar de nada.

Cada capítulo do guia corresponde a uma skill. Leia o capítulo, instale a skill, use.

PARA QUEM

Para quem toca uma operação — estoque, expedição, vendas, financeiro, obra, clínica, serviço — e enxerga um processo manual que custa horas por semana. Não é preciso saber programar. É preciso conhecer o processo melhor do que qualquer programador conheceria.

O QUE VOCÊ TERÁ NO FIM

Um aplicativo de uso interno no ar, instalado no celular da sua equipe, com os dados da empresa dentro, backup configurado e um manual de uma página. Não um protótipo: uma ferramenta que a equipe abre na segunda-feira.

COMO USAR ESTE MATERIAL

Na ordem, se o projeto começa agora. Pelo capítulo que dói, se o app já existe. Cada capítulo termina com um checklist — não avance sem ele fechado, porque a etapa seguinte apoia na anterior.

Sumário

ANTES DE COMEÇAR

Por que agora — o app de uso interno	4
Como falar com o assistente: 7 regras	6
O mapa das 10 skills	8

PARTE I · DESCOBRIR

01 Escopo de uma página	10
02 Ambiente zero	13
03 Stack mínima	15

PARTE II · CONSTRUIR

04 Modelo de dados	18
05 Primeira tela viva	21
06 Fluxo completo	24

PARTE III · CONFIAR

07 Quem entra e quem vê	27
08 Dados que já existem	30
09 Não quebra	33

PARTE IV · ENTREGAR

10 No ar e vivo	36
-----------------------	----

FECHAMENTO

Estudo de caso: um app de estoque de verdade	39
A Instalar as 10 skills	42
B Skills públicas que combinam com cada etapa	44
C Os 30 dias — plano de execução	45
D Glossário sem jargão	46



Por que agora

O app de uso interno é o lugar onde construir com IA deixa de ser demonstração e vira dinheiro. Vale entender por quê antes de abrir o terminal.

Toda empresa tem um pedaço do trabalho que roda em planilha, caderno, grupo de WhatsApp e memória de alguém. Não é desleixo — é economia. O sistema que resolveria custa quarenta mil reais e seis meses, e ninguém gasta isso para economizar duas horas por semana.

Então a planilha fica. E junto com ela ficam os custos invisíveis: a conferência que demora duas horas, o produto que sumiu e ninguém sabe quando, o pedido faturado errado, a pessoa nova que leva três semanas para aprender "como a gente faz aqui". Some tudo e o software que não valia quarenta mil reais estava, na verdade, custando bem mais que isso.

O que mudou não foi a economia — foi o preço da construção. Um assistente de código transforma descrição em software funcionando. E, de repente, a conta vira a favor de qualquer processo que doa um pouco.

**Software interno não precisa ser bom para todo mundo.
Precisa ser certo para seis pessoas.**

Onde o software de prateleira perde

Um ERP é feito para servir dez mil empresas. Por isso ele tem 400 campos, dos quais você usa 12, e não tem exatamente o campo que o seu negócio precisa. Ele resolve 80% e o resto vira planilha paralela — que é onde mora o seu problema.

O app interno não compete com o ERP. Ele mora no buraco que o ERP deixou:

O QUE O ERP FAZ BEM	O QUE SOBRA PARA O APP INTERNO
Guardar o cadastro, emitir nota, fechar o mês	A conferência física da mercadoria que chegou
Registrar o pedido	A separação na rua, no celular, sem sinal
Controlar o financeiro	A prestação de contas do vendedor no fim do dia
Manter o catálogo	A conferência de preço na gôndola pelo código de barras

Repare no padrão: tudo que sobra acontece **fora da mesa** — em pé, com uma caixa na mão, no corredor, na rua. É exatamente o tipo de coisa que software genérico não cobre e que quem trabalha ali sabe descrever em três frases.

A vantagem que você tem e o programador não tem

Existe um mal-entendido caro: achar que a parte difícil de fazer software é escrever código. A parte difícil sempre foi *saber o que construir*. Quantas casas tem o código. Se o conferente pode bipar duas vezes o mesmo item. O que fazer quando chega mercadoria sem nota. Por que a coluna "obs" tem três significados diferentes.

Um programador contratado leva semanas descobrindo isso e ainda entende errado. Você já sabe. Com um assistente de código, essa passa a ser a parte que importa — e é a sua.

O QUE NÃO MUDOU

A IA escreve o código; ela não decide o que vale a pena. Ela também não sabe que o depósito não pega Wi-Fi, que a Dona Marli não usa celular com tela pequena, e que a conferência tem que acabar antes das 17h porque o caminhão sai. Essas decisões continuam sendo suas — e são elas que separam um app usado de um app abandonado.

O que este material entrega

Dez etapas, dez skills. Uma skill é um arquivo de instruções que você instala no assistente; a partir daí, quando você descreve o problema, ele conduz a etapa do jeito certo — faz as perguntas que precisam ser feitas, produz o documento que precisa existir, evita a armadilha que derruba a maioria dos projetos.

É a diferença entre pedir ajuda a alguém que já fez isso vinte vezes e pedir ajuda a alguém muito inteligente que nunca fez.



Como falar com o assistente

Sete regras que separam quem tira duas horas de trabalho da IA de quem tira duas semanas de retrabalho. Vale para Claude, Codex, Gemini ou qualquer outro.

1. Descreva o problema, não a solução

A tentação é pedir a implementação: *"crie uma tabela com paginação e um modal de edição"*. Você acabou de decidir a solução — e você é quem menos sabe sobre soluções técnicas nessa conversa.

Peça o problema: *"o conferente precisa ver o que já bipou hoje e corrigir uma quantidade errada sem perder o resto"*. O assistente propõe a forma. Se não gostar, aí sim você dirige.

2. Uma coisa por vez

Pedido grande gera código grande, que ninguém revisa, que esconde três erros. Pedido pequeno gera código que cabe na tela, que você lê, que você entende.

Regra prática: se o pedido tem mais de um "e depois", quebre em dois.

3. Diga o que não fazer

Assistentes são generosos por padrão: você pede uma tela e recebe um framework. As frases que mais economizam tempo são as de restrição.

COLE ISSO NO FIM DOS SEUS PEDIDOS

Sem instalar biblioteca nova. Siga o padrão dos arquivos que já existem. Não refatore o que não pedi. Se precisar de uma decisão que eu não dei, pergunte antes de escrever código.

4. Exija o "por quê" antes do "como"

Quando o assistente propõe um conserto, pergunte **por que quebrou**. Se a explicação não fizer sentido para você, provavelmente o conserto está escondendo o sintoma — e o problema volta em duas semanas, disfarçado.

5. Nunca aceite "deve funcionar"

Rode. Abra. Clique. Bipe o código que não existe. Se você não viu com os próprios olhos, não funciona — e essa regra vale independentemente de quão convincente foi a explicação.

6. Trate erro como informação, não como fracasso

Cole a mensagem de erro inteira, sem editar, junto com o que você fez antes dela. A mensagem que parece um monte de letras embaralhadas costuma dizer o arquivo e a linha exatos. Editar o erro para "ficar mais limpo" é remover a informação útil.

7. Commit antes de tentar algo grande

Antes de pedir uma mudança arriscada, salve um ponto de restauração:

```
git add . && git commit -m "antes de mexer na conferencia"
```

Com isso, "tentar" fica barato. Sem isso, toda tentativa é uma aposta — e você começa a evitar mudanças por medo, que é como projetos morrem sem ninguém perceber.

A ARMADILHA CENTRAL

Código gerado por IA parece pronto. É bem formatado, comentado, com nomes bonitos — e às vezes está errado de um jeito que só aparece com dado real, na segunda-feira, com um funcionário parado na sua frente. A aparência de acabamento não é evidência de correção. É por isso que existe o capítulo 9.



O mapa das 10 skills

A jornada inteira em uma página. Cada linha é um capítulo, uma skill instalável e um entregável concreto.

SKILL	O QUE ELA FAZ	VOCÊ TERMINA COM
01 escopo-de-uma-pagina	Vira o processo manual em escopo, e corta o que não é v1	ESCOPO.md
02 ambiente-zero	Instala tudo na sua máquina e valida com um teste real	Máquina pronta + repositório
03 stack-minima	Escolhe a tecnologia mais simples que aguenta, e registra o porquê	DECISOES.md
04 modelo-de-dados	Transforma o processo em entidades, eventos e regras	MODELO.md
05 primeira-tela-viva	Põe no ar uma tela que faz uma coisa real com dado real	App publicado fazendo uma coisa
06 fluxo-completo	Cresce fatia por fatia e cobre vazio, erro, offline e desfazer	O processo inteiro funcionando
07 quem-entra-e-quem-ve	Login proporcional, papéis e segredos fora do navegador	App seguro o suficiente
08 dados-que-ja-existem	Importa planilha e ERP com validação, quarentena e relatório	Dados reais dentro do app
09 nao-quebra	Teste de fumaça, revisão adversarial e rollback barato	Rede de segurança
10 no-ar-e-vivo	Publica, instala no celular, faz backup e controla custo	Ferramenta que a equipe usa

A ordem importa (mas não é um trilho)

As dez etapas estão em ordem de dependência: o modelo de dados apoia na decisão de stack, que apoia no escopo. Pular para trás custa retrabalho, e o retrabalho fica mais caro conforme se avança — trocar o modelo de dados no capítulo 4 é um arquivo de texto; trocar no capítulo 10 é uma migração com o app no ar.

Mas ninguém começa exatamente no zero. Se o app já existe, entre pela dor:

"NINGUÉM SABE DIZER O QUE ELE FAZ"

Capítulo 1 — escreva o escopo do que já existe.

"QUEBRA A CADA MUDANÇA"

Capítulo 9 — instale a rede de segurança antes de continuar.

"O NÚMERO NÃO FECHA"

Capítulo 4 — o modelo guarda saldo em vez de guardar eventos.

"ESTÁ NO AR E NÃO TEM BACKUP"

Capítulo 10 — pare tudo e resolva hoje.

"TENHO A PLANILHA E NÃO SEI COMO ENTRA"

Capítulo 8 — importação com quarentena.

"A EQUIPE NÃO USA"

Capítulo 5 — a validação com o usuário real foi pulada.

Escopo de uma página

skill: escopo-de-uma-pagina

Antes de existir uma linha de código, precisa existir uma página que qualquer pessoa da empresa lê em dois minutos e concorda.

O problema

Quase todo app interno que morre pela metade morreu do mesmo jeito: começou pela tela. Alguém pediu "um sistema de estoque", o assistente gerou catorze telas bonitas, e três dias depois ninguém sabia dizer qual problema aquilo resolvia.

O custo não foi o código — foi a decisão que ninguém tomou. E o sintoma é sempre o mesmo: o projeto cresce em telas e não cresce em utilidade.

Se você não consegue dizer o que o app faz em uma frase, o assistente também não consegue.

Como a skill conduz

Ela não te deixa começar pela tela. Ela faz cinco blocos de perguntas, três por vez, e não avança enquanto o bloco anterior não fechar:

BLOCO	O QUE ELA ARRANCA DE VOCÊ
A dor	A tarefa manual, quem faz, quantas vezes, quanto tempo, e <i>um erro concreto que já aconteceu</i>
O usuário	O cargo (não "a equipe"), o aparelho e o ambiente — em pé no depósito é outro app que sentado no escritório
O fluxo	O caminho feliz em no máximo 7 passos, como receita
Os dados	O que precisa sobreviver ao fechar o app, de onde vem hoje, e quanto
Pronto	Uma frase mensurável de como você saberá que a v1 funcionou

Duas dessas perguntas fazem quase todo o trabalho. A primeira é "**que erro concreto já aconteceu?**" — se você não consegue citar um, o problema não dói o bastante para virar software, e a skill vai te dizer isso na cara. A segunda é "**em que aparelho, em que lugar?**", porque ela decide metade da arquitetura antes de qualquer discussão técnica.

COMO COMEÇAR

Quero parar de controlar a conferência de mercadoria na planilha e fazer um app que a expedição usa no celular. Me ajuda a escrever o escopo antes de a gente escrever código.

O entregável

Um arquivo `ESCOPO.md` na raiz do projeto com sete seções: a dor, usuário e contexto, o caminho feliz, dados que precisam sobreviver, pronto é quando, riscos — e a seção que decide o destino do projeto.

A seção mais importante é a que lista o que NÃO entra

Ela precisa ter no mínimo três itens, e eles precisam ser coisas que você *realmente quer*. Se tudo que ficou de fora for palha que ninguém pediu, o corte não foi feito — só foi encenado.

Candidatos clássicos a ficar de fora de uma v1:

- Relatórios e gráficos (a v1 exporta CSV; o gráfico vem quando alguém sentir falta)
- Notificação automática por WhatsApp ou e-mail
- Perfis de permissão granulares — na v1 bastam dois: quem administra e quem opera
- App nativo na loja de aplicativos: um site que abre no celular resolve
- Integração de mão dupla com o ERP — comece lendo; escrever de volta é outro projeto
- Histórico de auditoria completo

ARMADILHAS

- **Escopo que é lista de telas.** "Tela de cadastro, tela de listagem, tela de relatório" não é escopo, é chute de implementação. Escopo é fluxo e dado.
- **O app para todo mundo.** Se serve a três cargos diferentes, escolha o que sofre mais e escreva para ele.
- **Substituir o ERP.** Ninguém substitui o ERP num fim de semana. O app interno ganha onde o ERP é ruim: o pedaço manual que ninguém automatizou.
- **Escopo escrito sozinho.** Se a pessoa que vai usar o app todo dia não leu o `ESCOPO.md`, ele não está validado.

DO MUNDO REAL

Num app de conferência de recebimento, o escopo cabia em uma frase: *"o conferente bipa o código de barras de cada item que chegou e o app diz na hora qual produto é e quanto já foi contado"*.

Fora da v1 ficaram: cadastro de produtos (vem sincronizado do e-commerce), registro de devolução, relatório gerencial, e integração de escrita com o ERP. Três desses quatro foram pedidos com convicção na primeira conversa. Nenhum deles era necessário para resolver as duas horas de conferência — e o app ficou pronto porque ficaram de fora.

SÓ AVANCE QUANDO

- ☐ Existe um `ESCOPE.md` com as sete seções preenchidas
- ☐ O caminho feliz tem no máximo 7 passos
- ☐ A seção "FORA da v1" tem pelo menos 3 itens que você queria de verdade
- ☐ "Pronto é quando" é uma frase mensurável, com número
- ☐ A pessoa que vai usar o app leu e disse "sim, isso resolve"

Ambiente zero

skill: ambiente-zero

A maior taxa de desistência não está no código. Está nos primeiros quarenta minutos, no terminal preto que responde `command not found`.

O problema

Quem nunca programou não sabe que erro no terminal é normal. Acha que quebrou o computador, e para. É a etapa com maior abandono do processo inteiro — e a mais fácil de resolver, porque é só sequência.

O objetivo aqui não é "instalar coisas". É chegar a um estado verificável: **você digita um comando, algo acontece, e você entende o que aconteceu.**

Como a skill conduz

Um comando por vez. Nunca um bloco de seis. Ela diz o que vai acontecer antes de você apertar Enter — "esse comando baixa uns 80 MB, demora dois minutos e no fim imprime um número de versão" — e pede a saída antes de mandar o próximo.

São quatro instalações e um teste:

O QUÊ	PARA QUÊ	COMO CONFERIR
Node.js (versão LTS)	É onde o assistente roda	<code>node -v</code> responde um número
Git	É o "salvar com histórico" e o desfazer do projeto	<code>git --version</code>
O assistente	Claude Code, Codex CLI ou Gemini CLI	abre e responde
GitHub	Backup, histórico e publicação	repositório <i>privado</i> com 1 commit

```
npm install -g @anthropic-ai/claude-code    # Claude Code
npm install -g @openai/codex                # Codex CLI
npm install -g @google/gemini-cli           # Gemini CLI
```

OS DOIS ERROS QUE PEGAM TODO MUNDO

"Instalei e continua dizendo que não existe." O terminal só enxerga programas novos depois de ser fechado e aberto de novo. Feche a janela inteira e reabra.

`EACCES: permission denied no npm install -g`. Não resolva com `sudo` — isso instala como administrador e cria problemas piores depois. Mude a pasta global do npm:

```
mkdir -p ~/.npm-global && npm config set prefix ~/.npm-global  
echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.zshrc && source ~/.zshrc
```

O teste que prova que funciona

A skill não deixa a etapa fechar sem isso. Dentro da pasta do projeto, você pede:

O PRIMEIRO PEDIDO DE TODOS

Crie um arquivo `index.html` com uma página que mostre "Funciona" em letras grandes e a data de hoje. Depois me diga como abrir no navegador.

Quando a página abrir, você não instalou um ambiente — você viu o ciclo inteiro funcionar: **pedir → gerar → ver acontecer**. Todo o resto do material é esse ciclo repetido com pedidos maiores.

As três coisas que você precisa saber usar

Só isso, por enquanto. O resto vem quando doer.

1. `cd pasta` — entrar numa pasta (`cd ..` volta uma)
2. `git add . && git commit -m "o que mudou"` — salvar ponto de restauração
3. `git push` — mandar para o GitHub

E as duas de emergência: `git log --online` mostra o histórico, `git restore .` joga fora as mudanças não salvas quando algo der muito errado.

SÓ AVANCE QUANDO

- ☐ `node -v` e `git --version` respondem com números
- ☐ `git config user.email` mostra o seu e-mail
- ☐ O assistente abre e responde
- ☐ Existe um repositório **privado** no GitHub com pelo menos um commit
- ☐ Você abriu, no seu navegador, uma página que você mandou criar

Stack mínima

skill: stack-minima

Complexidade não é gratuita. Ela é paga depois, sozinho, no dia em que quebra.

O problema

Assistentes de código têm um viés previsível: eles geram o que mais apareceu no treino. Para "app de estoque", isso costuma significar uma stack que uma equipe de cinco pessoas manteria com esforço — entregue a uma pessoa que abriu o terminal semana passada.

Não é má-fé nem burrice do modelo. É que o material do mundo foi escrito por e para empresas de software, e você não é uma.

Escolha a coisa mais simples que aguenta o volume do seu escopo — não a mais simples que existe.

Simples demais também cobra: uma planilha compartilhada com oito pessoas editando ao mesmo tempo é um problema pior que um banco de dados.

As seis perguntas que decidem tudo

1. Quantas pessoas usam ao mesmo tempo?
2. Em que aparelho — celular, desktop, os dois?
3. Precisa funcionar sem internet?
4. Os dados podem ficar só no aparelho, ou várias pessoas precisam ver o mesmo dado?
5. Existe sistema que já tem esses dados?
6. Quem vai mexer nisso daqui a seis meses?

A sexta é a que mais gente ignora e a que mais cobra. Se a resposta é "ninguém", a stack precisa ser algo que continue funcionando *parado* — sem atualização de dependência, sem certificado expirando, sem servidor para reiniciar.

A tabela de decisão

SITUAÇÃO	STACK	POR QUÊ
1–3 pessoas, dados de leitura, roda no celular	HTML + CSS + JS puro no GitHub Pages	Zero build, zero servidor, zero custo
4–20 pessoas veem o mesmo dado	Front simples + Supabase	Banco e login prontos sem administrar servidor
Pouco estado compartilhado	Front + Cloudflare Workers e KV	Um arquivo de servidor, custo quase zero
Muitas telas, +20 pessoas	Next.js + Postgres gerenciado	Aguenta crescer — aceite o custo de manutenção
O dado vive numa planilha e vai continuar	App que lê e escreve na planilha	Não migre o que ninguém pediu para migrar
Só transformar dado e cuspir relatório	Não faça um app. Script + planilha	Nem todo problema é app

O default quando você ainda não sabe

Se a resposta honesta às seis perguntas for "não sei ainda", comece por: HTML, CSS e JavaScript sem framework e sem passo de build; dados no próprio aparelho; hospedagem no GitHub Pages; e um `manifest.json` que transforma o site em app instalável no celular — sem loja de aplicativos, sem taxa, sem revisão da Apple.

Isso não é gambiarra de iniciante. É a stack certa para o problema: um app usado por seis pessoas do estoque não tem os problemas de um app usado por seis milhões.

O filtro de dependências

Toda biblioteca nova passa por quatro perguntas:

1. Quanto código eu escreveria sem ela? Se a resposta for "umas 30 linhas", escreva as 30.
2. Ela exige um passo de build?
3. Ela quebra sozinha se ficar seis meses sem atualizar?
4. Se ela sumir amanhã, o app para?

Diga o número

A skill fecha com custo mensal estimado em reais, incluindo o cenário de dobrar o uso — e liga o alerta de gasto em qualquer serviço cobrado por uso *antes* de publicar. Conta inesperada de nuvem é a forma mais rápida de um projeto interno perder patrocínio.

ARMADILHAS

- **Escolher pela lista de vagas.** A stack que dá emprego não é a que resolve o problema de seis pessoas do estoque.
- **Banco de dados por reflexo.** Muito app interno guarda menos de 5 mil registros que mudam uma vez por dia. Isso cabe num arquivo versionado no Git.
- **Docker sem servidor.** Se você não vai administrar um servidor, você não precisa de container.
- **Login com provedor externo antes de existir app.** Configurar isso leva mais tempo que a v1 inteira.

SÓ AVANCE QUANDO

- ☐ Existe `DECISOES.md` com a stack, o porquê e o que foi recusado
- ☐ Você respondeu às seis perguntas — inclusive a sexta
- ☐ O custo mensal está escrito, com o cenário de dobrar o uso
- ☐ Está escrito qual gatilho concreto obriga a revisar essa decisão

Modelo de dados

skill: modelo-de-dados

Tela errada se conserta numa tarde. Modelo de dados errado se conserta migrando tudo, com o app no ar, com gente usando.

O problema

Esta é a única decisão do material que é cara de desfazer. E quase todo app interno quebrado tem o mesmo defeito de origem: modelaram a **tela** em vez de modelar o **fato**. Guardaram "linha da planilha" em vez de guardar "o que aconteceu".

O princípio: guarde eventos, calcule saldos

Veja o modelo que 90% das pessoas escreve primeiro:

```
produto: { sku, nome, quantidade_em_estoque }
```

O campo `quantidade_em_estoque` responde *quanto tem agora* e destrói todo o resto. Quando o número estiver errado — e vai estar — ninguém consegue descobrir por quê. Não dá para responder quem tirou, quando, de qual nota, quanto entrou ontem.

```
produto: { sku, nome, ean, ativo }  
movimento: { id, sku, tipo: entrada|saida|ajuste, quantidade,  
              quando, quem, origem, observacao }
```

Agora o saldo é uma conta: `soma(entradas) - soma(saídas) ± ajustes`. Você ganha histórico, auditoria e conserto de erro sem perder nada.

Não guarde o resultado. Guarde o que aconteceu, e calcule o resultado.

O raciocínio vale fora do estoque. Não guarde "status do pedido": guarde as mudanças de status. Não guarde "horas do funcionário no mês": guarde as batidas. Não guarde "saldo do cliente": guarde os lançamentos. Toda vez que alguém precisar responder "por que esse número está assim?", você terá a resposta.

Como a skill conduz

1. **Extrai os substantivos** do caminho feliz do `ESCOPO.md`.

- 2. Separa coisa de acontecimento.** Existe por si só? É entidade. Aconteceu com alguma coisa? É evento.
- 3. Procura a chave natural** — SKU, EAN, CPF, número da nota. Chave natural evita duplicata na origem. Mas cuidado com as que parecem estáveis e não são: nome de pessoa muda, e-mail muda.
- 4. Define os tipos com precisão** (a tabela abaixo).
- 5. Escreve as invariantes** — as regras que nunca podem ser violadas.
- 6. Gera dados de exemplo sujos** de propósito.

Os tipos que evitam bug de segunda-feira

DADO	COMO GUARDAR	PORQUE NÃO DO OUTRO JEITO
Dinheiro	inteiro em centavos	decimal com ponto flutuante erra na soma
Data	ISO 8601 com fuso	<code>20/08/2026</code> não ordena e não tem hora
Código de barras	texto , nunca número	zero à esquerda some e EAN-13 estoura a precisão de inteiro
Quantidade	inteiro, salvo se vende a granel	meia unidade que não existe vira bug silencioso
Sim/não	<code>ativo: true/false</code>	<code>status: "S"/"N"</code> vira três valores em seis meses

Dados de exemplo precisam ser sujos

Crie de 10 a 20 registros que *pareçam os reais*, com a sujeira real: nome com acento, descrição de 90 caracteres, SKU com hífen, um produto inativo, uma quantidade zerada, um campo opcional vazio.

Dados de exemplo limpos demais escondem exatamente os bugs que vão aparecer na segunda.

PEDIDO QUE FUNCIONA

Com base no ESCOPO.md, proponha o modelo de dados. Separe entidades de eventos, escolha chave natural onde existir, e escreva as invariantes. Depois gere `dados-exemplo.json` com 15 registros que pareçam dados reais de uma empresa brasileira — com acento, com um registro inativo e com um campo opcional vazio.

ARMADILHAS

- **Campo "observações" como lixeira.** Se três informações diferentes vão parar nele, são três campos.
- **Guardar o que dá para calcular.** Idade, total do pedido, saldo.
- **Apagar de verdade.** Em app de empresa, `ativo: false` quase sempre é melhor que apagar. Dado deletado por engano numa sexta não volta.
- **Traduzir metade.** `product.nome` e `movimento.date` no mesmo projeto custa uma hora de confusão por semana.
- **Modelar para o relatório.** Modele o fato; relatório é consulta.

SÓ AVANCE QUANDO

- ☐ Existe `MODELO.md` com entidades, eventos, relações e invariantes
- ☐ Nenhum campo guarda algo que dá para calcular a partir dos eventos
- ☐ Código de barras e documento estão como texto
- ☐ Existem 10–20 registros de exemplo com sujeira real
- ☐ Você pegou **três situações que já aconteceram** na empresa — inclusive uma que deu errado — e escreveu como cada uma fica registrada no modelo

Primeira tela viva

skill: primeira-tela-viva

Uma tela, um dado real, funcionando no celular de quem vai usar — hoje, não no mês que vem.

O problema

Existem dois jeitos de construir. O primeiro monta as doze telas, depois liga tudo, e no dia da ligação descobre que o modelo não fecha: três dias para jogar fora. O segundo faz **uma** coisa funcionar de ponta a ponta e só então cresce.

O segundo é mais rápido, e é o único que dá para fazer sozinho. Tem também um efeito humano que ninguém mede e que decide projetos: quem vê o próprio dado na própria tela em uma hora continua. Quem passa três dias em preparação, não.

A definição de "viva"

Cinco condições, sem exceção:

1. Abre no navegador (ou no celular) do usuário real, não só na sua máquina
2. Mostra **dado de verdade** da empresa — não *Produto 1*, *Produto 2*
3. Aceita **uma** ação do usuário, e a ação tem efeito visível
4. O efeito **sobrevive** a fechar e reabrir a página
5. Está commitada e publicada

Faltou uma? Não está viva — está bonita.

Escolha o corte onde a dor mora

Volte ao caminho feliz e implemente o passo que revela a ideia — não o primeiro passo.

APP	NÃO COMECE POR	COMECE POR
Estoque	cadastro de produtos	bipar um código e ver o nome do produto aparecer
Ordem de serviço	cadastro de clientes	abrir um chamado e ele aparecer na lista
Conferência de caixa	cadastro de formas de pagamento	lançar um valor e ver o total mudar

O cadastro é sempre a parte mais chata e menos reveladora. Faça depois.

O PEDIDO — REPARA NO NÍVEL DE DETALHE

Crie uma página única `index.html` com um campo que recebe um código de barras. Ao bipar, procure o código no `dados-exemplo.json` e mostre em letra grande o nome do produto e o saldo. Se não achar, mostre "não encontrado" em vermelho. Registre cada leitura no `localStorage` e mostre as últimas 5 embaixo. Sem framework, sem passo de build. Português no que aparece na tela.

O roteiro da sessão

1. Peça o corte, não o app.
2. Rode e olhe. `npx http-server . -p 8080` e abra no navegador. Não aceite "deve funcionar".
3. Teste com o dado que quebra. Um código que não existe. Um com zero à esquerda. Letra onde espera número. Vazio e Enter. Cada um desses é um bug que ia aparecer na segunda-feira com um funcionário na sua frente.
4. Commit.
5. Publique. GitHub Pages: Settings → Pages → `main`. Dois minutos e existe uma URL. Abra no celular.
6. Mostre para quem vai usar — hoje.

O passo 6 é o mais valioso do material inteiro

Leve a URL para o conferente, o vendedor, a pessoa do financeiro. Fique calado e olhe.

Você vai descobrir em cinco minutos coisas que não descobriria em duas semanas: que a letra é pequena demais para ler em pé, que o campo perde o foco depois de cada leitura, que ele precisa do preço e não do saldo, que o leitor manda um Enter que você não previu.

O QUE AINDA NÃO FAZER

Deliberadamente fora desta etapa: login, cadastro completo, menu e navegação, design além de "legível e com botões grandes", e tratamento de todos os erros possíveis. Tudo isso tem capítulo próprio. Fazer agora só atrasa a única coisa que importa nesta etapa: descobrir se a ideia funciona na mão de quem trabalha.

ARMADILHAS

- **Aceitar a tela de demonstração.** Se o assistente inventar dados dentro da própria página, peça de novo lendo do arquivo real — dado falso esconde erro de modelo.
- **Deixar para publicar depois.** "Funciona na minha máquina" não é entregável. Publicar na primeira sessão transforma deploy em rotina em vez de evento.
- **Crescer antes de validar.** Se você já está na terceira tela e ninguém usou a primeira, pare e mostre.

SÓ AVANCE QUANDO

- ☐ Existe uma URL que abre no celular
- ☐ A tela mostra dado real da empresa
- ☐ Uma ação funciona e o efeito sobrevive ao recarregar
- ☐ Pelo menos uma pessoa que não é você usou e comentou
- ☐ Você anotou o que ela disse — inclusive o que te incomodou ouvir

Fluxo completo

skill: **fluxo-completo**

O caminho feliz é 20% do trabalho e 80% das demonstrações. Quem trabalha em operação vive nas bordas.

O problema

O que separa um protótipo de uma ferramenta usada todo dia é o comportamento nas bordas: lista vazia, internet caiu, o código não existe, a pessoa clicou duas vezes, a bateria do tablet acabou no meio da conferência.

Cresça por fatias verticais

Uma fatia vertical é um passo do caminho feliz funcionando **inteiro**: tela, regra, gravação e feedback. Nunca faça "todas as telas" e depois "toda a lógica".

1. A ação que a pessoa mais repete (bipar, lançar, conferir)
2. A ação que corrige a anterior (ajustar, desfazer, editar)
3. A visão do que já foi feito (lista, resumo do dia, saldo)
4. A saída dos dados (exportar CSV, imprimir, mandar)
5. Só então: cadastro e configuração

Cada fatia é um commit e é testável por uma pessoa real.

Os sete estados obrigatórios de toda tela

ESTADO	COMO TESTAR	O QUE PRECISA APARECER
Vazio	usuário novo, banco zerado	frase que ensina o próximo passo
Carregando	DevTools → Network → Slow 3G	indicação visível; botão desabilitado
Erro	desligue o Wi-Fi e tente	o que houve, em português, e o que fazer
Sem permissão	entre com o usuário mais limitado	o botão some — não aparece e bloqueia depois
Muitos dados	5.000 registros	ainda rola liso; busca antes de lista
Duplo clique	clique 3x rápido em Salvar	grava uma vez
Desfazer	grave algo errado de propósito	dá para reverter sem mexer no banco

O estado vazio é o mais negligenciado e o mais barato de acertar:

✗ "Nenhum resultado encontrado."

✓ "Nenhuma conferência hoje. Bipe o primeiro código para começar."

Regra de ouro do app de operação: o dado não pode morrer

Se o app é usado em pé, no celular, com uma caixa na outra mão, ele *vai* perder conexão e a aba *vai* ser fechada por engano. Portanto:

- **Grave a cada ação**, não ao final. Um botão "Salvar" no fim da conferência é uma armadilha: quem esquece perde uma hora de trabalho.
- **Ao reabrir, ofereça retomar**: "você tem uma conferência de 47 itens iniciada às 14h32. Continuar?"
- **Sincronize quando houver rede**, com a pendência visível na tela.
- **Se dois aparelhos editarem a mesma coisa, detecte e avise** — nunca sobrescreva calado.

Esse trio — grava sempre, retoma, avisa conflito — é o que faz a operação confiar no app. Sem ele, basta uma perda de trabalho para a equipe voltar para o papel.

Feedback em ambiente barulhento

Visual não basta no depósito. Use som curto e *distinto* para sucesso e para erro (dois tons diferentes, não o mesmo), vibração no celular (`navigator.vibrate(50)`), e cor ocupando área grande em vez de um texto de 12px. Nunca use só `alert()` : trava a tela e exige toque.

PEÇA COMPORTAMENTO, NÃO COMPONENTE

Adicione a tela de resumo da conferência: lista os itens bipados agrupados por SKU, com quantidade, ordenados pelo último bipado no topo. Precisa ter estado vazio com a frase "Nenhum item bipado ainda", botão Exportar CSV desabilitado quando a lista está vazia, e desfazer no último item. Precisa continuar fluida com 3.000 itens. **Não crie framework novo, siga o padrão dos arquivos existentes.**

A última frase importa mais do que parece: sem ela, o assistente costuma introduzir uma biblioteca nova a cada pedido, e o projeto vira uma colcha de retalhos.

Arrumação a cada quatro fatias

Pare e peça um diagnóstico — sem deixar ele mudar nada ainda:

DIAGNÓSTICO

Leia os arquivos do projeto e me diga: (1) que código está repetido em mais de um lugar, (2) que função passou de 50 linhas, (3) que nome não corresponde ao que a função faz. **Não mude nada ainda, só me mostre a lista.**

Quinze minutos a cada quatro fatias. Deixar para o mês três custa uma reescrita.

ARMADILHAS

- **Pedir "termine o app".** Pedido grande gera código grande que ninguém revisa.
- **Confirmar tudo.** "Tem certeza?" em toda ação treina a pessoa a clicar sim sem ler. Confirme só o irreversível — e prefira oferecer desfazer.
- **Mensagem de erro técnica.** `TypeError: undefined` não ajuda ninguém. Traduza para o que a pessoa deve fazer.
- **Menu antes de conteúdo.** App com 4 telas não precisa de menu lateral com ícones; precisa de 4 botões grandes.
- **Formulário de 20 campos.** Se o processo real preenche 4, mostre 4.

SÓ AVANCE QUANDO

- ☐ Todo passo do caminho feliz funciona no app
- ☐ Toda tela tem estado vazio, de carregamento e de erro
- ☐ O trabalho em andamento sobrevive a fechar a aba
- ☐ Existe desfazer para a ação mais perigosa
- ☐ Uma pessoa da operação fez um ciclo completo **sozinha**, sem você do lado

Quem entra e quem vê

skill: quem-entra-e-quem-ve

Segurança proporcional: nem projeto corporativo de dois meses, nem a chave da API do ERP dentro do JavaScript.

O problema

App interno tem dois erros clássicos e opostos.

O primeiro é montar autenticação corporativa completa — provedor externo, dois fatores, recuperação por e-mail, papéis granulares — antes de existir app. Duas semanas gastas em algo que oito pessoas resolveriam com uma senha combinada.

O segundo, muito mais grave, é publicar na internet um app que carrega a **chave da API do ERP** dentro do JavaScript. Qualquer pessoa aperta F12 e leva as credenciais da empresa inteira.

A pergunta que decide o nível

Se a pessoa errada abrir isso, o que ela consegue fazer?

NÍVEL	RESPOSTA	O QUE BASTA
1	Ver dado que já está no site público	Nenhum login. Publique.
2	Ver custo, margem, dado de cliente	Login simples com lista de autorizados
3	Alterar estoque, preço ou pedido de verdade	Login + papéis + segredo no servidor
4	Ver dado pessoal de terceiros (LGPD)	O acima + registro de acesso + retenção

Suba um nível quando doer, não antes. Mas **nunca** fique abaixo do nível 3 se o app escreve em sistema de produção.

A regra inegociável: segredo não mora no navegador

Tudo que vai para o navegador é público. Sempre. Não existe "escondido no JavaScript", não existe "ofuscado", não existe "está numa variável".

```
// x NUNCA — qualquer usuário lê isso em 5 segundos
const API_KEY = "d11a7a789f97e0f2b8c4";
fetch("https://api.erp.com/produtos", { headers: { "X-Key": API_KEY } });
```

O jeito certo tem duas formas, e as duas custam poucos minutos.

FORMA A — UM INTERMEDIÁRIO MÍNIMO

Um arquivo de servidor guarda a chave, recebe o pedido do app, chama o ERP e devolve só o que interessa:

```
// worker/src/index.js — a chave vive aqui, no servidor
export default {
  async fetch(request, env) {
    const r = await fetch("https://api.erp.com/produtos", {
      headers: { "X-Key": env.ERP_API_KEY } // variável de ambiente
    });
    return new Response(await r.text());
  }
};
```

```
npx wrangler secret put ERP_API_KEY # guarda o segredo fora do repositório
```

FORMA B — CADA PESSOA USA A PRÓPRIA CREDENCIAL

O app pede a chave na primeira vez e guarda **só no aparelho**. Serve bem quando o sistema emite credencial por usuário e o app é usado por poucas pessoas.

SE A CHAVE JÁ ESTÁ NO CÓDIGO

Troque a chave. Não basta apagar do arquivo: ela está no histórico do Git para sempre, e o histórico vai junto quando alguém clona o repositório. Rotacionar a credencial é a única correção real.

Login proporcional: os três degraus

DEGRAU 1 — LISTA DE AUTORIZADOS

Uma lista de e-mails permitidos e senha por usuário. Vale para 3 a 15 pessoas de confiança, em app que *lê* dado sensível mas não escreve em produção.

Honestidade obrigatória aqui: se a lista está num arquivo que vai para o navegador, isso é uma **tranca, não um cofre** — impede o colega curioso, não impede quem sabe abrir o F12. Diga isso a quem pediu, em vez de vender segurança que não existe, e mantenha o dado sensível de verdade atrás do intermediário da Forma A.

DEGRAU 2 — LOGIN DE VERDADE SEM CONSTRUIR LOGIN

Um serviço pronto (Supabase Auth, Clerk, Auth0) entrega senha com hash, recuperação por e-mail, sessão e expiração sem você escrever nada disso. Uma a duas horas de integração.

DEGRAU 3 — ENTRAR COM A CONTA DA EMPRESA

Se a empresa usa Google Workspace ou Microsoft 365, "Entrar com Google", restrito ao domínio da empresa, resolve entrada e saída de funcionário de uma vez: desligou, perdeu o acesso. Não há senha para gerenciar.

Papéis: comece com dois

Não modele uma matriz de permissões. Comece com **operador** (faz o trabalho do dia) e **admin** (configura, corrige, vê o sensível). Adicione um terceiro só quando alguém for barrado de fato.

Esconder o botão é conforto.

A checagem no servidor é a segurança.

Esse é o buraco mais comum em app feito com IA: a tela esconde o botão, mas a rota do servidor aceita a chamada de qualquer um. Faça as duas coisas — sempre.

ANTES DE PUBLICAR

- ☐ Nenhuma chave, senha ou token em arquivo que vai para o navegador
- ☐ `git log -p | grep -iE "api.?key|secret|password|token"` não devolve segredo vivo
- ☐ `.gitignore` tem `.env`, e o `.env` nunca foi commitado
- ☐ Se algo vazou no histórico, a credencial foi **rotacionada**
- ☐ O repositório está **privado**
- ☐ Toda ação de escrita valida o usuário **no servidor**
- ☐ Existe caminho para tirar o acesso de alguém em menos de 5 minutos
- ☐ Se guarda dado pessoal: você sabe dizer qual, por quê e por quanto tempo

COMBINE COM

Antes de publicar, rode uma revisão de segurança dedicada — no Claude Code, `/security-review`. Ela pega o que checklist manual esquece: injeção, CORS aberto, dependência vulnerável, dado sensível caindo em log.

Dados que já existem

skill: dados-que-ja-existem

A pior importação não é a que falha. É a que "funciona" e cria quatro mil registros errados.

O problema

Nenhum app interno nasce em terreno limpo. O dado já existe — numa planilha que passou por seis pessoas, num ERP com campo usado para outra coisa, num CSV exportado com acento quebrado. E ele é sujo de um jeito específico e teimoso:

- A mesma unidade escrita como UN, Un, unid, unidade
- Preço como 1.234,56, 1234.56, R\$ 1.234,56 e 1234,56
- Código de barras que virou 7,89123E+12 porque alguém abriu no Excel
- Linhas em branco no meio, cabeçalho repetido na linha 500, um total no rodapé
- Duas linhas do "mesmo" produto: Caixa 20L e CAIXA 20 L

Importar é um processo com relatório, não um botão.

As cinco fases

1. RADIOGRAFIA — ANTES DE ESCREVER QUALQUER CÓDIGO

Nunca importe um arquivo que você não olhou.

ANTES DE IMPORTAR

Leia planilha.xlsx e me diga, sem alterar nada: quantas linhas; quais colunas; para cada coluna o tipo predominante, quantos vazios e 3 exemplos reais; quais colunas têm valor duplicado; e quais linhas parecem lixo (em branco, cabeçalho repetido, total).

Essa saída sozinha costuma revelar duas ou três surpresas.

2. MAPA DE-PARA

Um mapa explícito de coluna da planilha → campo do modelo. O que não estiver no mapa **não entra**.

COLUNA	CAMPO	TRANSFORMAÇÃO
Cód.	sku	trim, maiúsculas
Descrição do Produto	nome	trim, colapsar espaços
EAN	ean	texto; recompor zeros à esquerda; descartar notação científica
Preço Venda	preco_centavos	remover R\$, trocar separadores, ×100
Estoque	—	ignorado: saldo vem dos movimentos

3. NORMALIZAÇÃO

```
const texto = v ⇒ String(v ?? "").trim().replace(/\s+/g, " ");
const reais = v ⇒ Math.round(parseFloat(
  String(v).replace(/[R$\s.]/g, "").replace(",", ".") * 100));
```

Sobre o EAN em notação científica: **não tente reconstruir**. `7,89123E+12` perdeu dígitos de verdade. Marque como pendente e peça a exportação de novo com a coluna em formato texto. Inventar um dígito verificador é criar um erro que ninguém vai achar depois.

4. VALIDAÇÃO E QUARENTENA — A FASE QUE NINGUÉM PULA

Nada entra direto. Toda linha vai para um de três destinos: **aceita**, **quarentena** (precisa de decisão humana) ou **rejeitada**.

E sempre gere um relatório com uma linha por problema, contendo *o número da linha original*, o valor cru e o motivo. Sem isso, corrigir é impossível:

```
linha,sku,motivo,valor_original
147,TUT-CX-001,sku duplicado (ver linha 88),TUT-CX-001
203,,sku vazio,
418,TUT-PL-77,ean em notacao cientifica,7891234E+12
```

5. CARGA COM RESUMO E REVERSÃO

```
4.182 linhas lidas
3.940 aceitas
  198 em quarentena (147 duplicatas, 51 sem EAN)
  44 rejeitadas
Confirma a carga das 3.940? [s/N]
```

E grave de forma reversível: marque todos os registros com um `lote_id`. Se der errado, apagar o lote é um comando, não uma noite.

Duplicatas: decida a regra antes

Duplicata é decisão de negócio, não técnica. Mesma chave natural → mantém a primeira, manda as outras para quarentena. Nome parecido → **nunca funda automaticamente**; leve para quarentena com as duas linhas lado a lado.

Ninguém fica bravo com duplicata na quarentena. Todo mundo fica bravo quando dois produtos diferentes viraram um só.

Quando o dado vem de API

- **Pagine.** Quase toda API devolve no máximo 50–100 por página; importar sem paginar traz a primeira página e *parece* completo.
- **Respeite o limite de chamadas** — meio segundo entre requisições evita bloqueio.
- **Salve a resposta crua** antes de transformar.
- **Prefira sincronização incremental** — filtre por data de modificação em vez de baixar o catálogo inteiro toda vez.
- **Nunca chame a API do ERP direto do navegador** (capítulo 7).

ARMADILHAS

- **Importar direto em produção na primeira tentativa.** Rode em modo seco, olhe o relatório, depois carregue.
- **Confiar no total.** "Importou 4.000 de 4.182" quase sempre significa 182 sumindo calado. Exija que *aceitas + quarentena + rejeitadas* bata com o total lido.
- **Corrigir a planilha na mão e reimportar.** Corrija a *regra*: a planilha vai ser exportada de novo mês que vem com os mesmos defeitos.
- **Perder o arquivo original.** Guarde uma cópia intocada de tudo.

SÓ AVANCE QUANDO

- ☐ Existe um mapa de-para escrito e a regra de duplicata está decidida
- ☐ *Aceitas + quarentena + rejeitadas* = total de linhas lidas
- ☐ O relatório de importação foi revisado por quem conhece o dado
- ☐ Uma amostra de 10 registros foi conferida um a um contra a origem
- ☐ Dá para desfazer o lote com um comando

Não quebra

skill: nao-quebra

O app está no ar, funcionando. Uma mudança pequena quebra algo em silêncio. Ninguém percebe por três dias. É assim que projetos morrem.

O problema

Quando percebem, os dados de três dias estão errados, a confiança acabou e a equipe voltou para a planilha. Não adianta o app ser bom: confiança perdida em operação não volta com argumento, volta com meses de acerto.

Não é preciso cobertura de teste de software crítico. São precisos **três hábitos** que custam meia hora para instalar e valem o projeto inteiro.

Hábito 1 — Teste de fumaça dos caminhos críticos

Não teste tudo. Teste o que, se quebrar, *para a operação*. Em quase todo app interno são de três a sete caminhos. Num app de conferência:

1. Bipar um código existente mostra o produto certo
2. Bipar um código inexistente oferece associar, sem travar a bipagem
3. A conferência sobrevive a fechar e reabrir o navegador
4. Exportar CSV sai com o total certo
5. Usuário não autorizado não entra

PEDIDO

Escreva testes de fumaça para estes 5 caminhos usando Playwright. Um arquivo, um teste por caminho, sem simular o que dá para usar de verdade. Cada teste precisa falhar de forma óbvia, com mensagem em português dizendo qual caminho quebrou.

O teste que importa é o que já pegou um bug.

Toda vez que algo quebrar em produção, o conserto tem duas partes: corrigir e adicionar o teste daquele caso. Assim a mesma falha nunca volta duas vezes — e, sem esse hábito, ela volta.

Hábito 2 — Revisão antes de aceitar

Código gerado por IA parece pronto. É o risco principal do método inteiro.

```
git diff          # olhe você mesmo, sempre
```

REVISÃO ADVERSARIAL — O PEDIDO IMPORTA MAIS DO QUE PARECE

Revise o diff atual procurando **defeito**, não elogio. Para cada problema: arquivo, linha, qual entrada concreta faz quebrar e o que acontece. Foque em: dado que não existe, texto onde espera número, lista vazia, dois cliques, erro de rede, e valor já gravado no formato antigo. Se não houver defeito, diga "nenhum" — não invente.

No Claude Code, `/code-review` faz esse trabalho com verificação adversarial das descobertas, e `/security-review` cobre o lado de segurança.

AS TRÊS PERGUNTAS ANTES DE ACEITAR CÓDIGO QUE VOCÊ NÃO ENTENDEU

1. Se isso quebrar às 7h de uma segunda-feira, eu consigo achar onde?
2. Que dado eu tenho que **não** foi testado aqui?
3. Isso apaga ou sobrescreve alguma coisa? Dá para desfazer?

Se a resposta da primeira for "não", peça a explicação em português antes de aceitar. Código que ninguém entende é dívida com juros.

Hábito 3 — Commit pequeno, rollback barato

O Git é o desfazer do projeto, mas só funciona com commits pequenos e frequentes. Uma mudança por commit, mensagem que diz *o que mudou para o usuário* — não "ajustes".

```
git log --oneline          # ache o último commit bom
git revert <hash-do-commit-ruim> # desfaz mantendo o histórico
git push
```

`revert` é melhor que `reset` em código publicado: desfaz sem apagar história — e a história é o que permite entender o que aconteceu.

O momento perigoso: mudança no formato do dado

A quebra mais cara não é de código, é de **dado**. Você muda `quantidade` de inteiro para decimal, ou renomeia um campo, e os registros já gravados param de ser lidos. Antes de mexer em formato de dado gravado:

1. Faça o backup (capítulo 10)
2. Escreva a migração dos dados antigos e rode nela primeiro
3. Deixe o código ler os **dois** formatos por um tempo
4. Só remova o antigo quando não sobrar registro nele

A rotina semanal de 15 minutos

```
npx playwright test      # os caminhos críticos ainda passam?
git log --oneline -10     # o que mudou na semana
npm outdated              # dependência com falha conhecida?
```

E abra o app como usuário comum, no celular, e faça um ciclo completo. Leva três minutos e pega o que teste automático não pega: a tela que ficou feia, o botão que sumiu no iPhone, a lentidão que apareceu.

ARMADILHAS

- **Teste que testa o simulador.** Se o teste substitui o banco, a API e o relógio, ele testa a sua imaginação.
- **Cobertura como meta.** 80% de cobertura em código que ninguém usa não vale um teste no caminho que a operação percorre 200 vezes por dia.
- **Commitar sem rodar.** "É uma mudança pequena" é a frase que antecede todo incidente.
- **Consertar sem entender.** Se a explicação do conserto não fizer sentido, ele provavelmente esconde o sintoma.

SÓ AVANCE QUANDO

- ☐ Existem de 3 a 7 testes de fumaça nos caminhos que param a operação
- ☐ Os testes rodam com um comando e você sabe qual é
- ☐ Todo bug que apareceu em produção virou um teste
- ☐ Commits são pequenos, com mensagem que descreve o efeito
- ☐ Você já executou um `git revert` **de propósito**, para saber fazer

No ar e vivo

skill: no-ar-e-vivo

Publicar é onde o app deixa de ser projeto e vira ferramenta. Também é onde aparecem as perguntas que ninguém fez antes.

O problema

Quem instala no celular do pessoal? O que acontece se o tablet quebrar? Quem consegue mexer nisso se você estiver de férias?

Um app interno sem backup e sem manual não é um ativo da empresa. É um risco com a sua cara nele.

Publicar

Se o app é HTML, CSS e JS, o caminho mais barato é o GitHub Pages: Settings → Pages → Deploy from a branch → `main`. Em dois minutos existe uma URL, e a partir daí **todo** `git push` **publica**. Deploy deixa de ser evento.

Vale ir um passo além e carimbar a versão no deploy — economiza muito suporte:

```
- name: Carimbar versao
  run: sed -i "s#__VERSAO__#$(date +%Y.%m.%d)-${GITHUB_SHA::7}#g" index.html
```

Mostre essa versão num canto da tela. Quando alguém disser "não está funcionando", a primeira pergunta vira "qual versão aparece aí?" — e metade dos chamados morre em "recarregue a página".

SEGREDOS EM PRODUÇÃO

Nunca no código: no painel do provedor (Environment Variables) ou por linha de comando (`npx wrangler secret put ERP_API_KEY`). E confira o óbvio antes de dormir: `.env` no `.gitignore`, repositório privado.

Instalar no celular da equipe — sem loja de aplicativos

Um site vira app instalável com um arquivo na raiz:

```
{
  "name": "Estoque Bipagem",
  "short_name": "Estoque",
  "start_url": ".",
  "display": "standalone",
  "theme_color": "#1a1a1a",
  "icons": [{ "src": "icone-192.png", "sizes": "192x192", "type": "image/png" }]
}
```

Com `<link rel="manifest" href="manifest.json">` no cabeçalho, a instalação fica assim:

- **Android/Chrome:** menu : → "Instalar aplicativo"
- **iPhone/Safari:** botão compartilhar → "Adicionar à Tela de Início"

Vira ícone na tela, abre em tela cheia, sem barra de navegador. Sem loja, sem taxa de 30%, sem revisão da Apple — e a atualização chega sozinha no próximo acesso, o que resolve o pior problema de app corporativo: metade da equipe rodando a versão de três meses atrás.

Backup — a parte que ninguém faz

Responda por escrito às três perguntas. Se travar em alguma, o app ainda não está publicado de verdade:

1. Se o aparelho onde o dado está sumir agora, o que se perde?
2. Qual foi a última cópia dos dados e onde ela está?
3. Quanto tempo leva para voltar ao ar? Você já testou?

ONDE O DADO VIVE	BACKUP MÍNIMO
No próprio aparelho	botão "Exportar tudo (JSON)" + rotina semanal
Cloudflare KV	script que lista as chaves e salva em arquivo, semanal
Supabase / Postgres	backup do plano + um dump mensal guardado fora
Planilha na nuvem	histórico de versões — confirme que está ligado

Restauração não testada não é backup.

Escolha um dia, apague o dado de um ambiente de teste e restaure. A primeira vez sempre revela algo que faltava.

O manual de uma página

Escreva um `manual.html` que caiba numa tela e responda: para que serve; como instalar no celular (com print); os 3 a 5 passos do uso diário; os três erros mais comuns e a solução; quem chamar.

Na linguagem de quem usa — "bipe o código", não "efetue a leitura óptica do identificador". E acessível **dentro do app**, não num e-mail que se perde.

Manter vivo

A cada pedido da equipe, decida entre três destinos: entra agora, entra na lista, ou não entra — e diga por quê. Lista sem "não entra" vira backlog morto.

Uma vez por mês, 30 minutos: rodar `npm outdated` e os testes, conferir o custo, verificar se o backup rodou, e perguntar a uma pessoa que usa o que mais atrapalha hoje. Essa última pergunta gera as melhores tarefas do projeto.

BUS FACTOR

Escreva no `README.md`: onde está hospedado, quais contas, quem tem acesso, como publicar, como restaurar backup. Se só você sabe, o app é um risco para a empresa — e cinco parágrafos resolvem.

ESTÁ ENTREGUE QUANDO

- ☐ Existe URL estável e a equipe instala sozinha no celular
- ☐ Todo push publica (ou o comando está no `README.md`)
- ☐ A versão aparece em algum canto da tela
- ☐ Nenhum segredo no repositório; repositório privado
- ☐ Existe backup com data — e a restauração já foi testada
- ☐ Custo mensal conhecido, com alerta de gasto ligado
- ☐ Manual de uma página acessível dentro do app
- ☐ O `README.md` responde: onde está, quem tem acesso, como publicar, como restaurar



Estudo de caso

Um app de conferência de estoque construído por quem cuida da operação, não por uma equipe de software. O que as dez etapas produziram na prática.

Uma distribuidora pequena vendia por e-commerce e conferia mercadoria na mão. Duas pessoas, planilha aberta no notebook, caixa por caixa, cruzando descrição de produto com o que a nota dizia. Duas horas por recebimento, erro toda semana.

O que o escopo decidiu

Uma frase: *"o conferente bipa o código de barras de cada item que chegou e o app diz na hora qual produto é e quanto já foi contado"*. Pronto era quando a conferência saísse em 15 minutos.

Fora da v1: cadastro de produtos (vem do e-commerce), relatório gerencial, integração de escrita com o ERP. Três coisas pedidas com convicção na primeira conversa; nenhuma necessária para resolver as duas horas.

A stack que as seis perguntas produziram

Três pessoas, celular e tablet, **depósito sem Wi-Fi confiável**, e ninguém técnico para manter depois. As respostas eliminaram sozinhas metade das opções:

- HTML, CSS e JavaScript puro — sem framework, sem passo de build
- PWA instalável pelo navegador, sem loja de aplicativos
- Catálogo de produtos sincronizado para o próprio aparelho, para busca instantânea mesmo sem sinal
- GitHub Pages, publicando a cada push

As decisões de dado que salvaram o projeto

CÓDIGO DE BARRAS É TEXTO

A busca aceita variações de zero à esquerda — UPC-A de 12 dígitos e EAN-13 são o mesmo produto. Guardado como número, esse app não funcionaria.

SINCRONIZAÇÃO INCREMENTAL

Em vez de baixar o catálogo inteiro toda vez, filtra por data de modificação. Sync de minutos virou sync de segundos.

CACHE SÓ DO NECESSÁRIO

Id, SKU, código e nome ficam no aparelho.
Imagem e estoque são buscados sob demanda
— o que cabe no celular é o que precisa ser
instantâneo.

GRAVA A CADA BIPAGEM

Nunca "salvar no fim". A conferência sobrevive a
fechar o app, cair a bateria, trocar de aparelho.

O que só apareceu com o usuário real na frente

Nenhum dos três itens abaixo estava no plano. Todos vieram de ver alguém usando:

- 1. O leitor físico não digita num campo.** Leitores USB e Bluetooth se comportam como teclado: "digitam" muito rápido e terminam com Enter. O app passou a escutar o teclado global — sequência rápida terminada em Enter é bipagem, sem precisar de campo focado. Ninguém mais perde leitura porque tocou fora do campo.
- 2. Som importa mais que tela.** No depósito, com a caixa na mão, ninguém olha o celular a cada bipagem. Dois tons diferentes — um para achou, outro para não achou — resolveram mais que qualquer melhoria visual.
- 3. Código não encontrado não pode travar a fila.** A primeira versão parava e pedia decisão. Virou fluxo contínuo: registra, segue bipando, e a associação é feita no resumo, no fim.

Quando o dado precisou sair do aparelho

Uma necessidade real apareceu depois: começar a conferência no tablet e terminar no celular. Isso exigia estado compartilhado — e é exatamente o gatilho previsto no capítulo 3 para revisar a decisão de stack.

A resposta foi a menor possível: um único arquivo de servidor com um banco chave-valor, guardando a sessão de recebimento por um código, com expiração automática em 30 dias. Três rotas. O resto do app continuou client-side.

E, junto, a proteção que o capítulo 6 exige: ao salvar, o app manda a versão que ele acha que é a atual; se outro aparelho salvou por cima, o servidor recusa e devolve o conflito em vez de sobrescrever calado.

A LIÇÃO MAIS CARA ESTÁ NO HISTÓRICO

Existe no histórico do projeto um commit que adiciona registro de devolução com leitura de etiqueta por IA e geração de mensagem de WhatsApp. Duas semanas depois, existe outro commit que **remove** tudo isso.

Era impressionante e não era usado. Foi construído porque era interessante de construir, não porque alguém tinha o problema. Nada disso estava no escopo da v1 — e a única razão de ter entrado foi ninguém ter perguntado, na hora, se resolvia a dor descrita no capítulo 1.

Remover foi a decisão certa. Não ter feito teria sido melhor.

Onde as dez etapas apareceram

NO PROJETO REAL

01	Escopo de uma frase; três funcionalidades cortadas da v1
02	Node, Git e o assistente na máquina de quem cuida da operação
03	Sem framework, sem build, sem servidor — porque não havia quem mantivesse
04	Código de barras como texto; catálogo separado do que é buscado sob demanda
05	Bipar e ver o nome do produto — a primeira coisa que existiu
06	Doze telas, cada uma nascida de uma fatia; retomar sessão; fluxo contínuo
07	Lista de autorizados com senha individual; credenciais da loja só no aparelho do admin
08	Catálogo importado da API do e-commerce, com paginação e sync incremental
09	Commits pequenos e descritivos; a feature errada foi revertida sem trauma
10	GitHub Actions com versionamento automático; PWA instalável; manual em HTML

Duas horas viraram quinze minutos. Nenhuma linha desse app foi escrita por um programador contratado.

Instalar as 10 skills

Cinco minutos. Depois disso o assistente conduz cada etapa sozinho, sem você precisar lembrar de nada deste livro.

O que é uma skill, na prática

Uma skill é uma pasta com um arquivo `SKILL.md` dentro. O arquivo tem um cabeçalho que diz *quando* aquilo deve ser usado, e um corpo com as instruções.

Quando você descreve um problema, o assistente lê os cabeçalhos, reconhece que a situação bate com uma das skills, e passa a seguir aquelas instruções. Você não precisa chamar pelo nome.

É a diferença entre pedir ajuda a alguém muito inteligente e pedir ajuda a alguém que já fez isso vinte vezes.

Claude Code

Com a pasta `skills/` deste material baixada:

```
./instalar.sh
```

Ou manualmente, para valer em todos os seus projetos:

```
mkdir -p ~/.claude/skills  
cp -r escopo-de-uma-pagina ambiente-zero stack-minima modelo-de-dados \  
    primeira-tela-viva fluxo-completo quem-entra-e-quem-ve \  
    dados-que-ja-existem nao-quebra no-ar-e-vivo ~/.claude/skills/
```

Para instalar só no projeto atual, copie para `.claude/skills/` dentro da pasta do projeto. Confira com `/skills` — as dez devem aparecer.

Codex CLI, Gemini CLI e outros

Esses assistentes ainda não carregam skills automaticamente. Use o conteúdo como instrução: abra o `SKILL.md` da etapa e cole o corpo no início da conversa, ou junte tudo num arquivo `AGENTS.md` na raiz do projeto — vários assistentes leem esse arquivo sozinhos.

Como usar no dia a dia

BASTA DESCREVER O PROBLEMA

Quero parar de controlar o estoque na planilha e fazer um app para a equipe usar no celular.

Se quiser forçar uma etapa específica, chame direto: *"use a skill modelo-de-dados no meu caso"*.

AJUSTE AO SEU NEGÓCIO

As skills são arquivos de texto — leia e edite. Se a sua empresa tem uma regra fixa ("todo relatório sai em CSV com ponto e vírgula", "nunca guardamos CPF"), escreva dentro da skill correspondente. A partir daí o assistente respeita a regra em todos os projetos, sem você repetir. Isso é o que transforma este material no *seu* método em vez do meu.

Skills públicas que combinam

As dez skills deste material cuidam do método. Estas, já publicadas e disponíveis, cuidam da execução — e são gratuitas.

Assistentes modernos vêm com skills próprias, e algumas encaixam exatamente nas etapas deste livro. Use as duas coisas juntas: a nossa conduz a etapa, a pública faz o trabalho pesado dentro dela.

ETAPA	SKILL PÚBLICA	PARA QUÊ
01	<code>skill-creator</code>	Transformar o processo da sua empresa numa skill própria, depois que ele estabilizar
02	<code>init</code>	Gerar o <code>CLAUDE.md</code> do projeto, que dá contexto permanente ao assistente
02	<code>session-start-hook</code>	Preparar o repositório para rodar testes e verificações sozinho a cada sessão
03	<code>claude-api</code>	Se o app for usar IA por dentro: modelos, preço, limites e cache
04	<code>xlsx</code>	Ler a planilha atual para extrair o modelo que já existe na prática
05	<code>run</code>	Subir e dirigir o app para ver a mudança funcionando de verdade
06	<code>frontend-design</code>	Dar identidade visual em vez do visual genérico padrão
06	<code>dataviz</code>	Quando chegar a hora dos gráficos — e só então
07	<code>security-review</code>	Revisão de segurança antes de publicar. Rode sempre
08	<code>xlsx</code>	Importar, limpar e reexportar <code>.xlsx</code> , <code>.csv</code> e <code>.tsv</code>
09	<code>code-review</code>	Revisão adversarial do diff, com verificação das descobertas
09	<code>simplify</code>	Faxina de repetição e complexidade acumulada
10	<code>pdf / docx</code>	Gerar o manual do usuário em formato que a equipe abre
10	<code>loop</code>	Rotina recorrente: conferir backup, rodar testes, checar custo

Como saber o que você já tem

No Claude Code, `/skills` lista o que está instalado. Vale olhar antes de pedir para construir algo do zero: muita coisa que parece trabalho de programação já é uma skill pronta.

Os 30 dias

Um plano de execução em ritmo de quem tem um trabalho e faz isso nas brechas. Uma hora por dia útil.

DIAS	O QUE FAZER	TERMINA COM
1	Escopo (cap. 1). Só conversa e um arquivo de texto	<code>ESCOPO.md</code> lido por quem vai usar
2	Ambiente (cap. 2)	Uma página sua aberta no seu navegador
3	Stack (cap. 3) e modelo (cap. 4)	<code>DECISOES.md</code> e <code>MODELO.md</code>
4-5	Primeira tela viva (cap. 5)	URL no celular de alguém da operação
6-7	Ouvir, anotar, corrigir. <i>Não construa nada novo</i>	Lista do que realmente falta
8-14	Fatias verticais (cap. 6), uma por dia	Caminho feliz inteiro funcionando
15-17	Dados reais (cap. 8)	Dados da empresa dentro do app, validados
18-19	Acesso e segredos (cap. 7)	Checklist de publicação fechado
20-21	Rede de segurança (cap. 9)	Testes de fumaça rodando
22-23	Publicar e instalar (cap. 10)	Equipe com o ícone no celular
24-25	Backup, custo e manual	Restauração testada de verdade
26-30	Uso real com você por perto. Corrigir o que aparecer	A frase de "pronto é quando" cumprida

As três regras do plano

- 1. Os dias 6 e 7 são inegociáveis.** A vontade de pular a semana de ouvir e ir direto construir é forte, e é o erro mais caro do calendário inteiro.
- 2. Um dia perdido não é um dia a recuperar.** Empurre tudo um dia. Apertar o cronograma sempre sai do teste ou do backup — as duas coisas que você mais vai querer ter.
- 3. Se atrasar, corte escopo, não etapa.** Um app menor que passa por todas as dez etapas é infinitamente melhor que um app completo sem backup e sem teste.

Glossário sem jargão

O suficiente para você entender o que o assistente está dizendo — e nem uma palavra a mais.

Repositório (repo)	A pasta do seu projeto com histórico de todas as mudanças.
Commit	Um ponto de restauração salvo, com uma frase dizendo o que mudou.
Push	Mandar seus commits para o GitHub. É o backup do código.
Branch	Uma linha paralela para experimentar sem estragar o que funciona.
Deploy	Publicar: colocar a versão nova no ar para as pessoas usarem.
Front / back	Front é o que roda no aparelho da pessoa; back é o que roda no servidor.
API	A porta pela qual um sistema conversa com outro.
Endpoint / rota	Um endereço específico dessa porta. Ex.: <code>/produtos</code> .
JSON	Formato de texto em que os sistemas trocam dados.
localStorage	Uma gavetinha dentro do navegador do aparelho. Some se limpar o navegador.
PWA	Um site que instala como app no celular, sem passar por loja de aplicativos.
Framework	Um conjunto grande de código pronto que impõe um jeito de fazer as coisas.
Build	Etapas que transformam o código que você escreve no que o navegador entende.
Dependência	Código de terceiros que seu app usa. Cada uma é uma promessa de manutenção.
Variável de ambiente	Configuração guardada fora do código — é onde senhas e chaves devem morar.
Migração	Converter dados já gravados para um formato novo.
Diff	A lista exata do que mudou: linha adicionada, linha removida.
Rollback / revert	Voltar para uma versão anterior que funcionava.
Teste de fumaça	Verificação rápida de que o essencial ainda funciona.
Token / chave de API	Uma senha que identifica seu app perante outro sistema. Nunca vai no navegador.
Cache	Cópia guardada perto para não buscar de novo. Rápido — e às vezes desatualizado.
Terminal	A janela preta onde você digita comandos. Errar nela é normal.



O primeiro app é o mais caro

Não em dinheiro — em desconforto. É o primeiro em que você vai errar o modelo de dados, descobrir que o escopo estava grande, ouvir de alguém da sua equipe que a tela está confusa.

Todos os erros deste livro estão aqui porque alguém os cometeu. Você vai cometer alguns mesmo assim, e vai reconhecê-los mais rápido por ter lido.

O segundo app leva um terço do tempo. O terceiro deixa de parecer um projeto e passa a parecer o que é: uma ferramenta que a sua empresa fabrica quando precisa, do tamanho que precisa, do jeito que o processo realmente funciona.

É esse o ponto. Não é fazer *um* app. É a empresa deixar de esperar que alguém de fora resolva um problema que ninguém de fora entende.

**Comece pelo processo que mais te irrita hoje.
É quase sempre o certo.**

O melhor software interno é escrito por quem conhece o processo.

Durante trinta anos isso foi impossível: quem conhecia o processo não sabia programar, e quem sabia programar não conhecia o processo. O tradutor entre os dois custava caro, demorava meses e entendia metade.

Esse intervalo fechou. A pessoa que confere a mercadoria, que fecha o caixa, que monta a rota — essa pessoa agora consegue construir a ferramenta de que precisa.

Continua sendo trabalho. Só deixou de ser um trabalho de outra pessoa.